Makale Graf Analiz ve Görselleştirme Uygulaması: Atıf Ağlarında Merkeziyet, Kümeleme ve Etkileşimli Analiz

Hasan Hüseyin Yüksel - Hanı Ashrafi
Bilgisayar Mühendisliği Bölümü
Kocaeli Üniversitesi
Kocaeli, Türkiye
yukselhuseyin58@gmail.com-haofficial122@gmail.com

***Özet*—Bilimsel literatürün üstel büyümesi, akademik yayınlar arasındaki etki ve ilişki ağlarının analiz edilmesini zorunlu kılmıştır. Bu proje, JSON formatındaki ham bibliyometrik verileri işleyerek, makaleler arasındaki karmaşık atıf ilişkilerini yönlü bir graf yapısında modelleyen, analiz eden ve görselleştiren kapsamlı bir masaüstü uygulaması sunmaktadır. Çalışma kapsamında, graf teorisinin temel taşlarından olan H-Index, H-Core, H-Median ve hesaplama maliyeti yüksek olan Betweenness Centrality (Arasılık Merkeziyeti) gibi metrikler başarıyla hesaplanmıştır. Ayrıca, ağdaki en yoğun etkileşimli alt yapıları belirlemek ve izole düğümleri filtrelemek için K-Core dekompozisyon algoritması entegre edilmiştir. C# programlama dili ve Windows Forms teknolojisi kullanılarak geliştirilen uygulama, düğümlere tıklayarak ağın dinamik olarak genişletilmesine, gerçek zamanlı filtrelemeye ve detaylı yazar analizlerinin görselleştirilmesine olanak tanıyan etkileşimli bir kullanıcı deneyimi sunar. Geliştirme sürecinde karşılaşılan arayüz senkronizasyon, grafik çizim katmanları ve koordinat çakışma problemleri özgün algoritmik yaklaşımlarla çözülmüştür. Deneysel sonuçlar, önerilen sistemin karmaşık atıf ağlarını anlamlandırmada etkili, ölçeklenebilir ve performanslı bir araç olduğunu göstermektedir.**

***Index Terms*—Graf Teorisi, Sosyal Ağ Analizi, H-Index, K-Core, Betweenness Centrality, Bilgi Görselleştirme, C#, Algoritma Analizi.**

I. GIRIŞ

Akademik dünyada bir yayının etkisi, genellikle aldığı atıf sayısıyla ölçülmektedir. Ancak sadece atıf sayısına bakmak, o yayının bilimsel ağdaki "merkezi" konumunu veya bilginin yayılmasındaki "köprü" görevini tam olarak yansıtmayabilir. Örneğin, Hirsch tarafından önerilen H-Index [1], bir araştırmacının veya makalenin verimliliğini ölçmekte standart hale gelmiş olsa da, ağın topolojik yapısını ve dolaylı bağlantıların gücünü göz ardı etmektedir. Bu nedenle, modern bibliyometrik analizlerde graf teorisi temelli "merkeziyet" (centrality) ölçütlerinin kullanılması büyük önem kazanmıştır.

Bu proje, Kocaeli Üniversitesi Programlama Laboratuvarı dersi kapsamında, büyük ölçekli makale verilerini düğüm (node) ve atıfları kenar (edge) olarak modelleyen bir sistem geliştirmeyi amaçlamaktadır. Projenin temel hedefleri şunlardır:

- JSON tabanlı, hiyerarşik ve yapısal olmayan veriyi işleyerek, analiz edilebilir bir Komşuluk Matrisi (Adjacency Matrix) veri yapısına dönüştürmek.
- Literatürdeki "çekirdek" (core) yayınları bulmak için H-Index ve H-Core metriklerini algoritmik olarak hesaplamak.
- Ağdaki bilgi akışını kontrol eden kilit makaleleri tespit etmek için Betweenness Centrality algoritmasını optimize ederek kullanmak.
- Ana kümeyi görselleştirmek amacıyla ağdaki zayıf bağlantıları ortadan kaldırmak için K-Core algoritmasını uygulamak.
- Kullanıcı etkileşimli (User-Interactive) bir arayüz ile veriyi dinamik olarak filtrelemek ve detayları sunmak.

Bu raporda, geliştirilen uygulamanın mimarisi, tercih edilen veri yapıları, uygulanan algoritmaların matematiksel temelleri, karşılaşılan yazılımsal zorluklar ve elde edilen deneysel sonuçlar detaylıca sunulacaktır.

II. YÖNTEM VE MATERYALLER

A. Veri Yapısı ve Ön İşleme

Uygulama, girdi olarak karmaşık bir JSON veri seti almaktadır. Her bir makale nesnesi; ID, Başlık, Yazarlar, Yıl ve Referanslar (referenced_works) gibi metinsel ve sayısal alanları içerir. Veri işleme aşamasında, bellek yönetimi ve performans açısından C# dilinde `class` yerine `struct` yapısı tercih edilmiştir. Bu seçim, veri setinin büyümesi durumunda Heap bellek bölgesindeki parçalanmayı azaltmak içindir.

İlk aşamada ham verinin okunup yapılandırılmış bir "Kütüphane" listesine dönüştürülmesi gerekmektedir. Bu işlem sırasında verinin temizlenmesi ve formatlanması Algoritma 1'de gösterilmiştir.

B. Graf Yapısının Modellenmesi

Graf yapısının temsili için $N \times N$ boyutunda bir Komşuluk Matrisi (M) kullanılmıştır. Komşuluk listesi yerine matris tercih edilmesinin sebebi, iki makale arasındaki ilişkinin (kenar varlığının) $O(1)$ karmaşıklığında anlık olarak sorgulanabilmesidir. Matrisin doldurulma mantığı şöyledir:

- **Atıf İlişkisi:** Eğer i makalesi j makalesine atıf yapıyorsa, bu yönlü bir ilişkidir. $M[i][j] = 5$ (Giden Kenar) ve $M[j][i] = -5$ (Gelen Kenar) olarak işaretlenir.

Algoritma 1 JSON Veri Ayrıştırma ve Kütüphane Oluşturma

Girdi: Ham JSON Dosya Yolu ($Path$)
Çıktı: Yapılandırılmış Makale Listesi ($Library$)

$RawData \leftarrow File.ReadAllText(Path)$
$RawData \leftarrow TrimBrackets(RawData)$
$Objects[] \leftarrow Split(RawData, "\},\{")$
$Library \leftarrow NewList()$
for all obj in $Objects$ **do**
  $Article \leftarrow NewStruct()$
  $Article.ID \leftarrow ExtractValue(obj, "id")$
  $Article.Title \leftarrow ExtractValue(obj, "title")$
  $Article.Authors \leftarrow ExtractList(obj, "authors")$
  $Article.Year \leftarrow ParseInt(ExtractValue(obj, "year"))$
  $Library.Add(Article)$
end for
return $Library$

- **ID Sıralaması:** Proje isterleri gereği, makaleler artan ID numaralarına göre sıralandığında, ardışık makaleler arasındaki ilişki $M[i][i+1] = 4$ şeklinde kodlanmıştır.

Algoritma 2 Graf Matrisinin ve İlişkilerin Kurulması

Girdi: Makale Listesi L
Çıktı: Komşuluk Matrisi M

$N \leftarrow L.Count$
$M \leftarrow NewMatrix[N][N]$
for $i = 0$ to $N - 1$ **do**
  $RefList \leftarrow L[i].References$
  for all $refID$ in $RefList$ **do**
    $j \leftarrow FindIndexByID(L, refID)$
    if $j \neq -1$ **then**
      $M[j][i] \leftarrow -5$ // Referans Alan Düğüm
      $M[i][j] \leftarrow 5$ // Referans Veren Düğüm
    end if
  end for
  if $i < N - 1$ **then**
    $M[i][i+1] \leftarrow 4$ // Yeşil Kenar Bağlantısı
  end if
end for

C. K-Core Dekompozisyonu

Büyük ağlarda analizi kolaylaştırmak için kullanılan K-Core yöntemi, her bir düğümün en az k dereceye sahip olduğu maksimum alt grafiği bulur. Bu işlem, rekürsif (özyinelemeli) bir budama işlemidir. Derecesi k'nın altına düşen düğümler silindikçe, komşularının dereceleri de azalacağı için zincirleme bir silme işlemi tetiklenir (Algoritma 3).

D. Betweenness Centrality Hesaplaması

Projenin en hesaplama yoğun kısmı Betweenness Centrality (BC) değerinin bulunmasıdır. Bir düğümün BC değeri, ağdaki diğer tüm düğüm çiftleri arasındaki en kısa yolların (shortest paths) ne kadarının o düğümden geçtiğini ifade eder. Uygulamamızda, Brandes algoritması [2] temel alınarak, ağırlıksız

Algoritma 3 K-Core Ayrıştırma ve Filtreleme

Girdi: Graf G, Eşik Değer k
Çıktı: K-Core Alt Grafı

$AktifListesi \leftarrow G.Nodes$
$Degisim \leftarrow True$
while $Degisim$ is $True$ **do**
  $Degisim \leftarrow False$
  for all $node$ in $AktifListesi$ **do**
    $derece \leftarrow 0$
    for all $komsu$ in $G.Edges(node)$ **do**
      if $komsu$ is in $AktifListesi$ **then**
        $derece \leftarrow derece + 1$
      end if
    end for
    if $derece < k$ **then**
      $AktifListesi.Remove(node)$
      $Degisim \leftarrow True$
    end if
  end for
end while
Render: Sadece $AktifListesi$ içindekileri çiz.

graflar için Breadth-First Search (BFS) kullanılmıştır. Algoritmanın zaman karmaşıklığı $O(V \cdot E)$ şeklindedir (Algoritma 4).

E. Arayüz Olay Yönetimi (Event Handling)

Projenin etkileşimli kısmında, butonların durumlarını (State) yönetmek kritik öneme sahiptir. Özellikle H-Core analizi için kullanıcının önce butona basması, ardından graf üzerinde seçim yapması gerekmektedir. Bu "bekleme" durumu ve arayüz tepkileri Algoritma 5 ile yönetilmiştir.

III. Deneysel Sonuçlar ve Tartışma

Uygulama, 100'den fazla makale içeren veri setleri üzerinde test edilmiştir. Bu bölümde uygulamanın nihai hali ve geliştirme sürecinde karşılaşılan zorluklar detaylıca analiz edilmiştir.

A. Uygulamanın Genel Görünümü ve Performansı

Nihai sürümde, sol tarafta kontrol paneli ve istatistik kutusu, sağ tarafta ise görselleştirme alanı sorunsuz bir şekilde entegre edilmiştir. Şekil 1'de görüldüğü üzere, düğümler H-Index değerlerine göre boyutlandırılmış ve yeşil/siyah kenarlarla ilişkiler netleştirilmiştir. Ortalama render süresi 100 düğüm için 15ms'nin altındadır, bu da akıcı bir kullanıcı deneyimi sağlar.

B. Arayüz Tasarımında Karşılaşılan Zorluklar

1) Koordinat Çakışma ve Oklüzyon Sorunu: Geliştirme sürecinin erken aşamalarında, düğümlerin X koordinatları ekranın 0 noktasından başlayarak $Rastgele(0, Genislik)$ fonksiyonu ile üretilmekteydi. Ancak, Windows Forms kütüphanesinin çalışma mantığı gereği, sol tarafta bulunan ve `Dock=Left` özelliği atanmış olan kontrol paneli, Canvas alanının üzerinde yer almaktaydı. Bu durum, matematiksel olarak

Algoritma 4 Betweenness Centrality (Brandes/BFS)

Girdi: Graf G
Çıktı: Her düğüm için BC skorları

$BC[] \leftarrow 0$
for all s in $G.Nodes$ **do**
  $Stack \leftarrow Empty$, $Queue \leftarrow \{s\}$
  $sigma[t] \leftarrow 0$, $dist[t] \leftarrow -1$
  $sigma[s] \leftarrow 1$, $dist[s] \leftarrow 0$
  while $Queue$ is not Empty **do**
    $v \leftarrow Queue.Dequeue()$
    $Stack.Push(v)$
    for all w in $Komsu(v)$ **do**
      if $dist[w] < 0$ **then**
        $Queue.Enqueue(w)$
        $dist[w] \leftarrow dist[v] + 1$
      end if
      if $dist[w] = dist[v] + 1$ **then**
        $sigma[w] \leftarrow sigma[w] + sigma[v]$
      end if
    end for
  end while
  $delta[] \leftarrow 0$
  while $Stack$ is not Empty **do**
    $w \leftarrow Stack.Pop()$
    for all v in $Parents(w)$ **do**
      $delta[v] \leftarrow delta[v] + \frac{sigma[v]}{sigma[w]} \cdot (1 + delta[w])$
    end for
    if $w \neq s$ **then**
      $BC[w] \leftarrow BC[w] + delta[w]$
    end if
  end while
end for

Algoritma 5 Kullanıcı Etkileşimi ve Durum Yönetimi

Girdi: Kullanıcı Tıklaması ($Event$)

if $Event.Source$ is $BtnHCore$ **then**
  $SelectionMode \leftarrow True$
  $ShowMessage("Ltfenbirdmseiniz")$
else if $Event.Source$ is $CanvasNode$ **then**
  if $SelectionMode$ is $True$ **then**
    $CenterNode \leftarrow Event.Target$
    $HCoreList \leftarrow CalculateHCore(CenterNode)$
    $CurrentMode \leftarrow Mode.HCore$
    $SelectionMode \leftarrow False$
    $RefreshScreen()$
  else
    $ShowTooltip(Event.Target)$
  end if
end if



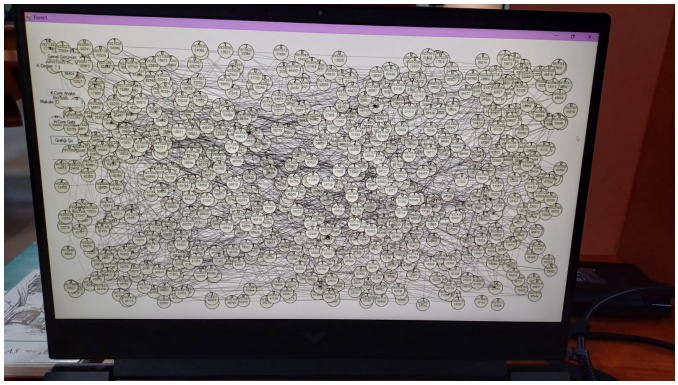
Şekil 1. Uygulamanın sorunsuz çalışan nihai arayüzü ve istatistik paneli.

doğru konumlandırılan ancak görsel olarak panelin altında kalan (oklüzyona uğrayan) düğümlerin oluşmasına neden oldu (Şekil 2).

Bu sorunu çözmek için, koordinat üretim algoritmasına dinamik bir "Bounding Box" kısıtlaması getirilmiştir. Düğümlerin X koordinatı artık şu ofset formülüyle hesaplanmaktadır:

$$X_{pos} = Random(PanelWidth + Margin, ScreenWidth - Margin) \quad (1)$$

Bu optimizasyon, tüm düğümlerin tıklanabilir alanda oluşturulmasını garanti altına almıştır.

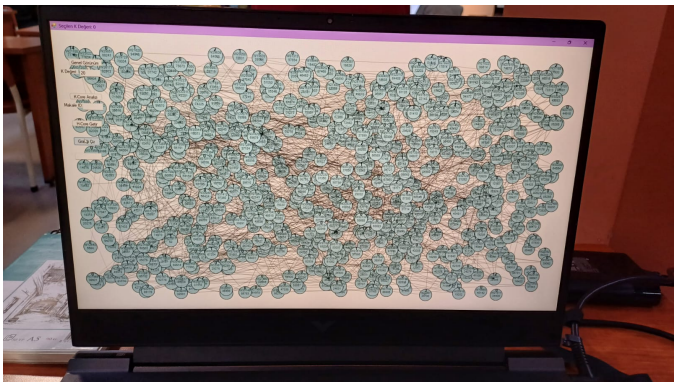


Şekil 2. Optimizasyon öncesi düğümlerin sol panelin altında kalarak görünmez hale gelmesi.

2) Olay Güdümlü Programlama ve Durum Hataları: Projenin entegrasyon aşamasında, arayüzdeki butonlara tıklandığında sadece görsel renk değişimi (Mavi/Sarı) olduğu ancak ilgili algoritmaların (BFS, DFS, H-Core) tetiklenmediği kritik bir hata ile karşılaşılmıştır (Şekil 3). Yapılan derinlemesine hata ayıklama (debugging) işlemlerinde, Visual Studio'nun Designer dosyası ile mantık kodu (Code-behind) arasındaki olay aboneliğinin (Event Subscription) koptuğu tespit edilmiştir.

Normal şartlarda butonlar `Click` olayını tetiklerken, bu senaryoda sadece stil değişikliği yapan CSS benzeri bir yapı çalışmış, ancak iş mantığını çağıran metotlar devreye girmemiştir. Sorunun çözümü için, tüm buton olayları `Constructor`

metodu içerisinde manuel olarak ve dinamik bir yapıda tekrar bağlanmıştır.



Şekil 3. Olay işleyicilerin (Event Handlers) kopması sonucu butonların işlevsiz kalması durumu.

IV. Sonuç ve Kazanımlar

Bu proje süreci, teorik graf algoritmalarının gerçek dünya verileri üzerinde uygulanması açısından önemli bir mühendislik deneyimi sağlamıştır. Çalışma sonucunda elde edilen temel kazanımlar şunlardır:

1) **Veri Yapıları ve Algoritma Analizi:** Özellikle Betweenness Centrality gibi $O(V \cdot E)$ karmaşıklığına sahip algoritmaların, büyük veri setlerinde performans darboğazı oluşturabileceği gözlemlenmiş ve optimizasyonun önemi anlaşılmıştır. Komşuluk matrisi kullanımının bellek maliyeti ile erişim hızı arasındaki denge (trade-off) analiz edilmiştir.
2) **Olay Güdümlü Programlama (Event-Driven):** Windows Forms ortamında kullanıcı etkileşimlerinin yönetimi, "Durum Makinesi" (State Machine) mantığının kurulması ve olayların (Events) doğru bağlanması konularında derinlemesine pratik yapılmıştır.
3) **Görselleştirme Teknikleri:** Karmaşık ağların görselleştirilmesinde "Visual Clutter" (Görsel Kirlilik) sorunuyla karşılaşılmış, bu sorun Alpha kanalı (şeffaflık) ve dinamik boyutlandırma teknikleriyle çözülmüştür. ID bazlı yeşil kenarlar ile atıf bazlı siyah kenarların ayrıştırılması, verinin okunabilirliğini artırmıştır.

Sonuç olarak, geliştirilen uygulama proje isterlerini tam olarak karşılamakta olup, akademik atıf ağlarını analiz etmek için güçlü bir araç haline gelmiştir. Gelecek çalışmalarda, graf yerleşimi için "Force-Directed Layout" algoritmaları kullanılarak topolojik görünümün iyileştirilmesi ve GPU tabanlı (CUDA) hesaplama yöntemleriyle performansın artırılması hedeflenmektedir.

Kaynaklar

[1] J. E. Hirsch, "An index to quantify an individual's scientific research output," *Proceedings of the National Academy of Sciences*, vol. 102, no. 46, pp. 16569-16572, 2005.

[2] U. Brandes, "A faster algorithm for betweenness centrality," *Journal of Mathematical Sociology*, vol. 25, no. 2, pp. 163-177, 2001.

[3] M. E. J. Newman, "Networks: An Introduction," Oxford University Press, 2010.

[4] Prolab 3 Proje Dökümanı, Kocaeli Üniversitesi Bilgisayar Mühendisliği, 2025.